PyAim

1. Project Overview

This project aims to develop a PyAim game using Python and Pygame. In this game, the player must click on appearing targets as quickly and accurately as possible before they disappear. The game is designed to improve reaction speed, mouse control, and aiming accuracy. The main objective of the game is to achieve the highest score possible by clicking targets within a limited time.

2. Project Review

A similar concept can be found in aim training games used in FPS games, where players practice their mouse accuracy and reaction speed. However, most aim trainer games do not provide detailed statistical analysis of player performance.

Key mechanics in the original concept:

- *Fast target appearance*
- *Quick player response*
- *Accuracy-based scoring*
- *Time pressure*

Improvements in this project:

1. ***Statistical Data Collection —*** *The game records detailed click data for every target interaction.*
2. ***Performance Analysis —*** *The player's reaction time, hit accuracy, and score will be analyzed.*
3. ***Difficulty Levels —*** *The game supports multiple difficulty settings by adjusting target size, speed, or spawn rate.*
4. ***Data Visualization —*** *The collected data will be displayed using graphs and charts.*
5. ***Object-Oriented Design —*** *The program is developed using Object-Oriented Programming principles, including inheritance.*

3. Programming Development

3.1 Game Concept

The PyAim game is a 2D reaction-based training game where players must click on targets before they disappear.

Game Mechanics:

1. *A target appears on the screen at a random position.*
2. *The player must click the target before it disappears.*
3. *If the player clicks successfully, they receive points.*
4. *If the player misses or the target disappears, it counts as a miss.*
5. *The game runs for a fixed amount of time.*
6. *The game records player actions and performance data.*

Game Objective

The main objective is to click as many targets as possible with high accuracy and fast reaction time.

Key Features

- *Random target spawning*
- *Real-time mouse input*
- *Score system*
- *Timer system*
- *Accuracy calculation*
- *Data recording*
- *Difficulty levels*

3.2 Object-Oriented Programming Implementation

The game is built around six core classes. Each class has a clearly defined responsibility following OOP principles.

Class	Purpose	Attributes	Methods
Game	Controls game loop and state	score, time_left, running, difficulty	start_game(), update(), draw(), end_game()
Player	Tracks player performance	total_clicks, hits, misses	register_hit(), register_miss(), get_accuracy()
Target	Represents targets (inherits GameObject)	x, y, radius, spawn_time	spawn(), draw(), is_clicked()
Timer	Manages game time	start_time, duration	get_time_left(), is_time_up()
ScoreManager	Handles scoring	score	add_score(), reset(), get_score()
DataCollector	Records gameplay data	records, filename	record_click(), save_to_csv(), load_data()

Inheritance Design:
A base class **GameObject** defines shared properties such as position (x, y) and a draw() interface. The **Target** class inherits from GameObject, reusing these attributes and extending them with target-specific behaviour such as spawn_time, radius, and collision detection. This design reduces code duplication and makes it straightforward to add new on-screen objects in the future.

3.3 Algorithms Involved

1. Random Position Generation

Targets appear at random positions on the screen. The algorithm generates random x and y coordinates and checks that the target fits inside the screen boundaries before placing it.

2. Collision Detection

The game checks whether the player's click lands on the target by comparing the distance between the click position and the target centre against the target radius. A hit is registered if the distance is less than or equal to the radius.

3. Time-Based Event Handling

Timing logic controls how long the overall game lasts and how long each individual target remains visible before disappearing.

4. Score Calculation

The score increases by a fixed amount each time the player successfully clicks a target. The ScoreManager class handles this logic.

5. Accuracy Calculation

Accuracy is computed from recorded hit and miss data after the game session ends using the formula: ***Accuracy = (Number of Hits / Total Clicks) × 100****. This is an aggregated statistic derived from raw per-click records.*

6. Data Recording

The game saves raw gameplay records into a CSV file in append mode, ensuring data from multiple sessions accumulates rather than being overwritten.

7. Statistical Visualization

The recorded data is visualised using graphs after gameplay sessions, allowing players to review their performance trends.

4. Statistical Data (Prop Stats)

4.1 Data Features

The game records raw per-click and per-target data to analyse player behaviour and performance. Accuracy is an aggregated statistic and is therefore not a raw feature; it is calculated in the analysis stage (see Section 4.3).

Features	Why it is useful	How to obtain 100+ data	Variable source	Display method
Reaction Time	Measures player response speed and skill level	Every click generates one reaction time record	Target.spawn_time, click_time (Target & Player)	Mean, Std Dev in summary; Histogram (Graph 1)
Hit/Miss Result	Evaluates success rate per click	Every click is recorded as hit or miss	register_hit(), register_miss() from Player	Hit/miss ratio in summary; Pie Chart (Graph 3)
Target Position (x, y)	Analyse whether certain screen zones are harder	Every target spawn includes position data	x, y from Target class	Bar Charts by screen zone (Graph 5)
Target Size	Compare difficulty and performance by target size	Every target has a radius value	radius from Target class	Mean per difficulty in summary; Boxplot (Graph 4)
Score	Measures overall player performance over time	Recorded per click or per round	score from ScoreManager	Mean, Max in summary; Bar Chart (Graph 2)
Click Timestamp	Analyse timing patterns and behaviour over time	Every click includes a timestamp	System time via Timer	Line Graph (Graph 4)

4.2 Data Recording Method

*Gameplay statistics are stored in a CSV file. Each click or target event generates one row of data. The file is opened in **append mode** so that records from multiple gameplay sessions accumulate instead of being overwritten. Even a short 30-second round generates 30–50 click records; a few sessions will easily exceed the 100-record threshold.*

Example CSV fields:

round, target_x, target_y, target_size, spawn_time, click_time, reaction_time, result, score, difficulty

4.3 Data Analysis Report

*The recorded data will be analysed using Python data analysis tools — specifically **pandas** and **matplotlib**. Accuracy is calculated at this stage from the raw hit/miss records using the formula: **Accuracy = (Hits / Total Clicks) × 100**. It is not stored as a raw feature because it is derived from other fields.*

Feature	Min	Max	Mean	Std Dev	Notes
Reaction Time (ms)	-	-	-	-	Calculated from all recorded clicks
Score per Round	-	-	-	-	Calculated across all rounds
Target Size (radius)	-	-	-	-	Compared across difficulty levels
Hit Rate (%)	-	-	-	-	Hits / Total Clicks × 100
Targets per Minute	-	-	-	-	Total hits / game duration in minutes

4.4 Data Visualization

At least five different graphs will be produced after each gameplay session, covering distribution, comparison, proportion, and time-series analysis.

Graph	Feature Name	Graph Objective	Graph Type	X-axis	Y-axis
Graph 1	Reaction Time	Show distribution of player reaction speed	Histogram (Distribution)	Reaction Time (ms)	Frequency
Graph 2	Score per Round	Compare performance across rounds	Bar Chart (Proportion/ Comparison)	Round Number	Score
Graph 3	Hit vs Miss	Show success proportion	Pie Chart (Proportion)	-	Hit / Miss Percentage
Graph 4	Score Progression	Show performance improvement over time	Line Graph (Time-series)	Time (s)	Score
Graph 5a	Target Position X Zone	Compare hit frequency across horizontal screen zones	Bar Chart (Comparison)	Screen Zone (Left / Centre / Right)	Hit Count
Graph 5b	Target Position Y Zone	Compare hit frequency across vertical screen zones	Bar Chart (Comparison)	Screen Zone (Top / Middle / Bottom)	Hit Count

5. Project Planning Timeline

Week	Week	Task
1	8	Proposal submission / Project initiation
2	9	Game structure and class design
3	10	Target, Player, and Timer system implementation
4	11	Data collection system
5	12	Statistics and graph visualisation
6	13	Testing and improvement
7	14	Submission week (Draft)

6. Document version

Version: 3.0
Date: 4 April 2026
Editor: Panuwitch Sowkasem
Student ID: 6810545867